Boréal Marché

Customer Intelligence Platform

Which customers should we call before they stop buying?

Quels clients faut-il joindre avant qu'ils cessent d'acheter?

Manish Chauhan (ID 6401170) | 420-977-VA AI Development Project, Sect. 91401

Live: bo-18e314a8dc3544148369c0305041cb8f.ecs.ca-central-1.on.aws

Code: github.com/AI-DS-Git/boreal-marche-customer-intelligence

Demo first (3 minutes)

Démo en direct

All on the **live service** on AWS (Canada, HTTPS), not on my laptop.

- **Prediction:** customer 12347 scores 81.0% retained, so no offer is suggested; the three main factors appear next to the score
- **Basket:** add one product and see the recommendation and *why* (the rule that fired)
- **Assistant, in French:** a policy question answered in French, with its source; a request for another customer's data is refused
- **Monitoring:** the page is green on normal traffic and turns red when I send drifted traffic
- **Version:** /version shows the commit that is running

Backup if the network fails: the 60–90 second README video of the same path.

The problem and what I built

Le problème

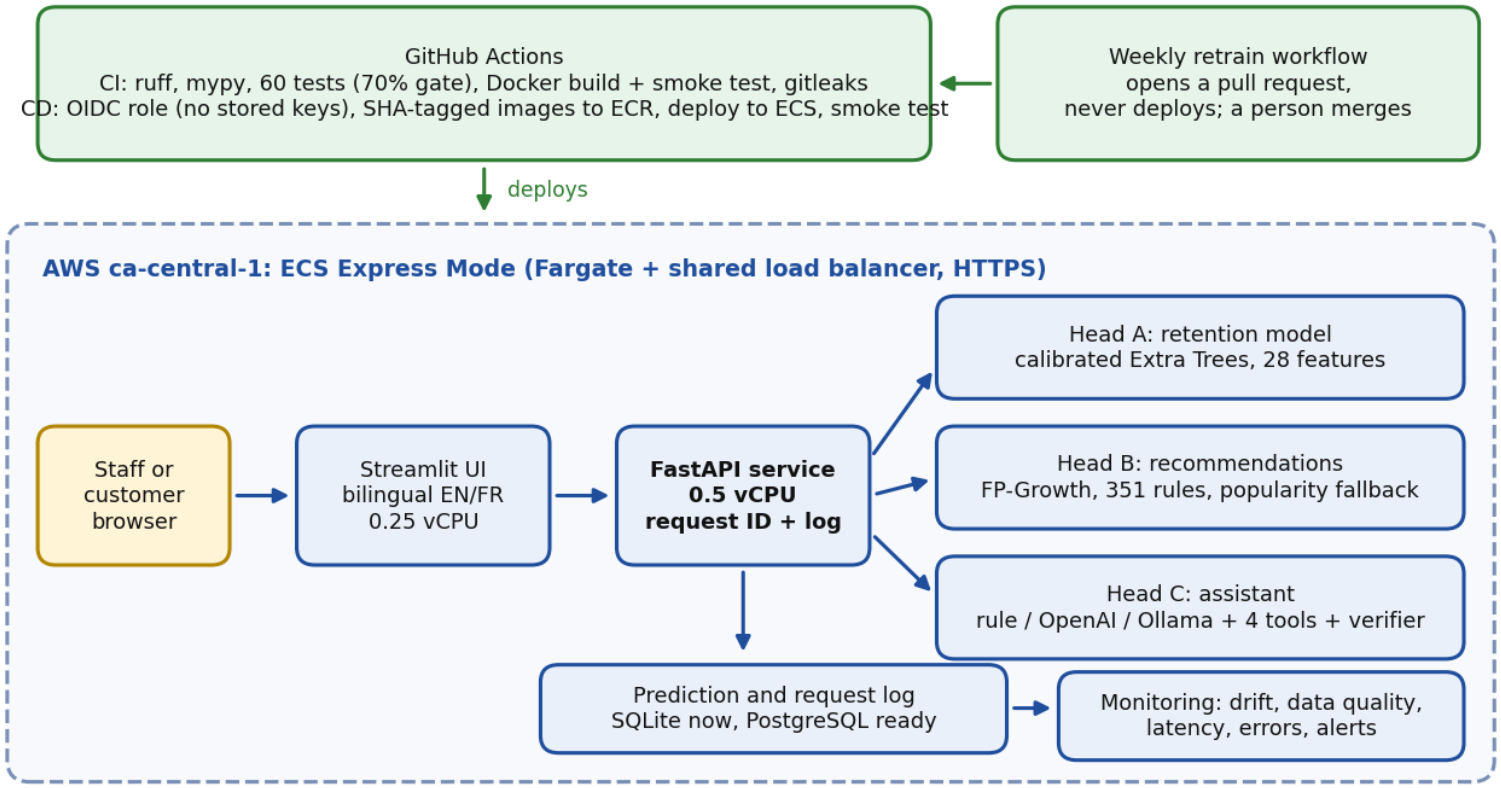
- Today every customer gets the same coupon each quarter. Most go to people who would have ordered anyway; the ones drifting away are not singled out.
- Questions arrive in French and English and are answered by hand.

Three AI components, one service

Head	Question it answers	Method
A: Retention	Who is unlikely to buy again in the next 90 days?	Calibrated Extra Trees, 28 features, profit-based threshold
B: Recommendations	What goes with this basket?	FP-Growth association rules, popularity fallback
C: Assistant	What is the policy? What is this product?	Retrieval + four typed tools + a verifier, EN/FR

Data: public UCI Online Retail II presented as Boréal Marché's. No real customer data. It is decision support: a person approves every offer.

Architecture

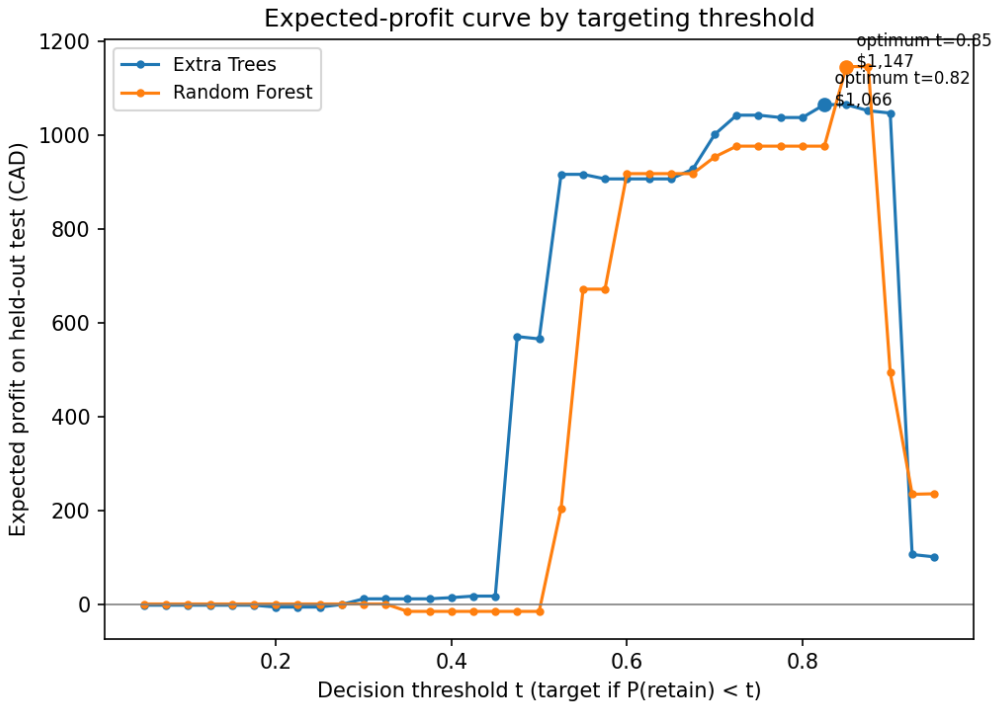


Choices that matter

- One artifact bundles preprocessing, model and calibrator
- Loaded once at start-up; /ready only when it works
- OIDC deploy, no stored AWS keys
- Weekly retrain opens a pull request and never deploys
- ECS Express instead of App Runner (closed to new customers; Streamlit needs WebSockets)

Hard decision: where to put the threshold

La décision difficile



Offer review if P(retain) < **0.82**, not 0.5.

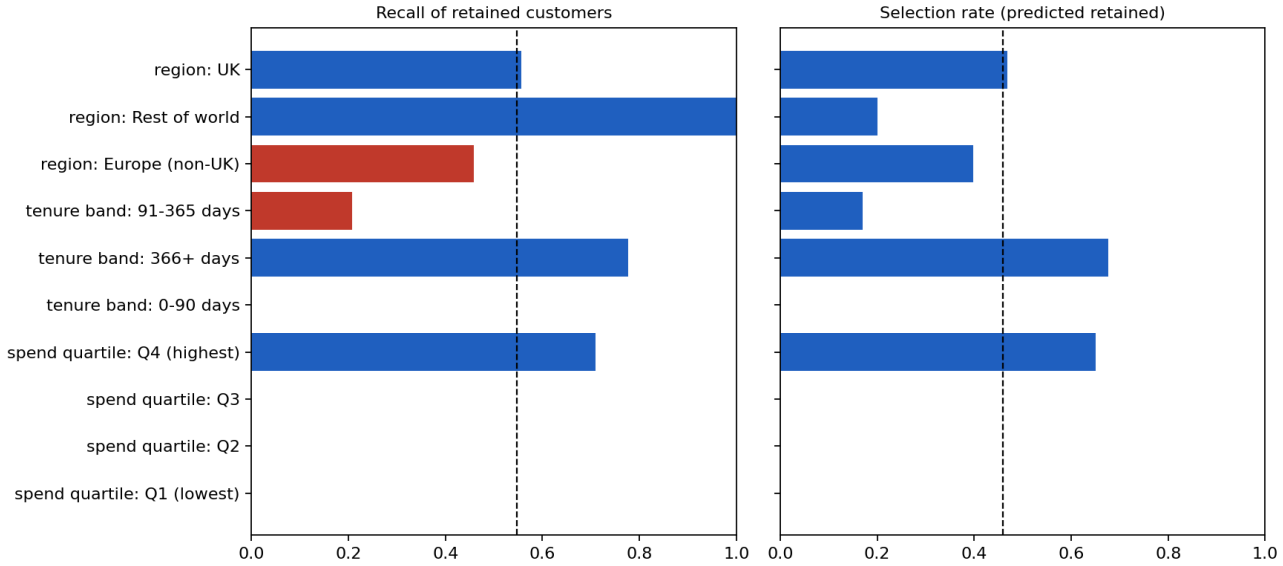
- A coupon costs $5. A saved customer is worth margin on a whole order. A wasted coupon is cheap, a lost customer is not.
- So the profit-maximising cut-off is high, and more customers are flagged than accuracy alone would suggest.
- Held-out result (788 customers): targeted **$1,067**, blanket coupon **$235**, no action $0.
- About **$1.06 per customer per quarter**, roughly $22,000 a year for 5,249 customers.

Honest caveat: coupon $5, margin 20%, recovery 30% are my assumptions. Changing one at a time keeps the gain positive; changing all three together loses to the blanket coupon in **5 of 27** scenarios. This is why I recommend a pilot with a control group.

Curve from the D3 run (its own split).

What that decision does to different customers

Équité entre groupes



The model never scores new or low-spend customers above 0.52, so **all** of them fall under 0.82.

- 6 of 10 groups are flagged (recall gap over 5 points)
- Spend quartiles Q1–Q3 and tenure 0–90 days: recall 0%, offer rate 100%
- Yet 41–66% of them were retained: about half of those offers are wasted

Mitigation	Result
A. Group thresholds	Worst-group recall 0% → 38%, precision 91% → 83%: pilot only
B. Rank within group	Spreads the review load; overall recall 55% → 51%
C. Recalibrate; D. Reweight	**Did not work**: the features lack the information

Decision: keep 0.82 and mandatory human review; use B to order the queue; collect features that separate new and low-spend customers.

Results against baselines

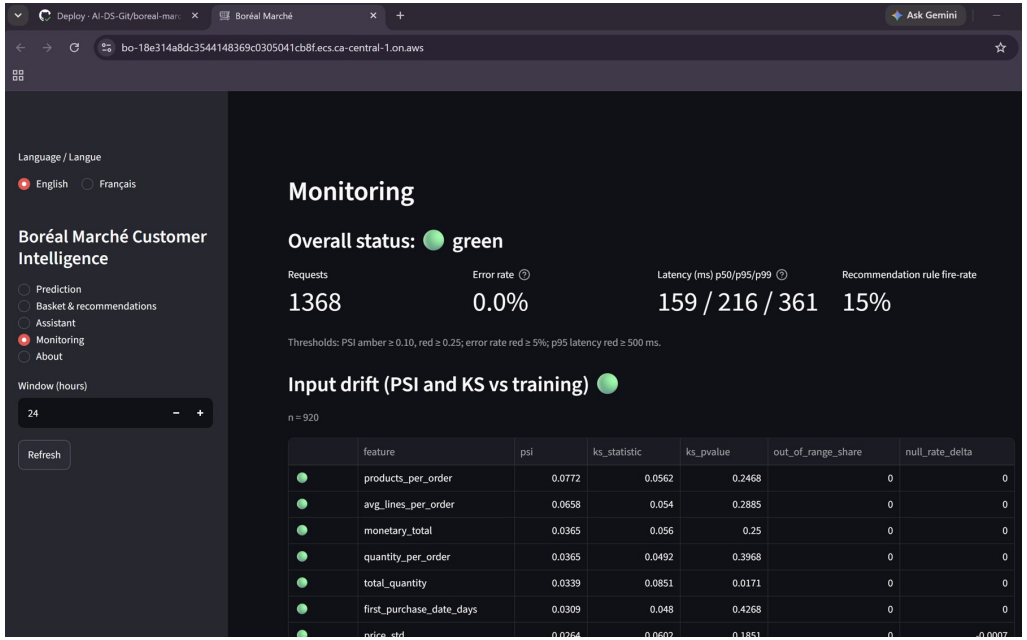
Résultats

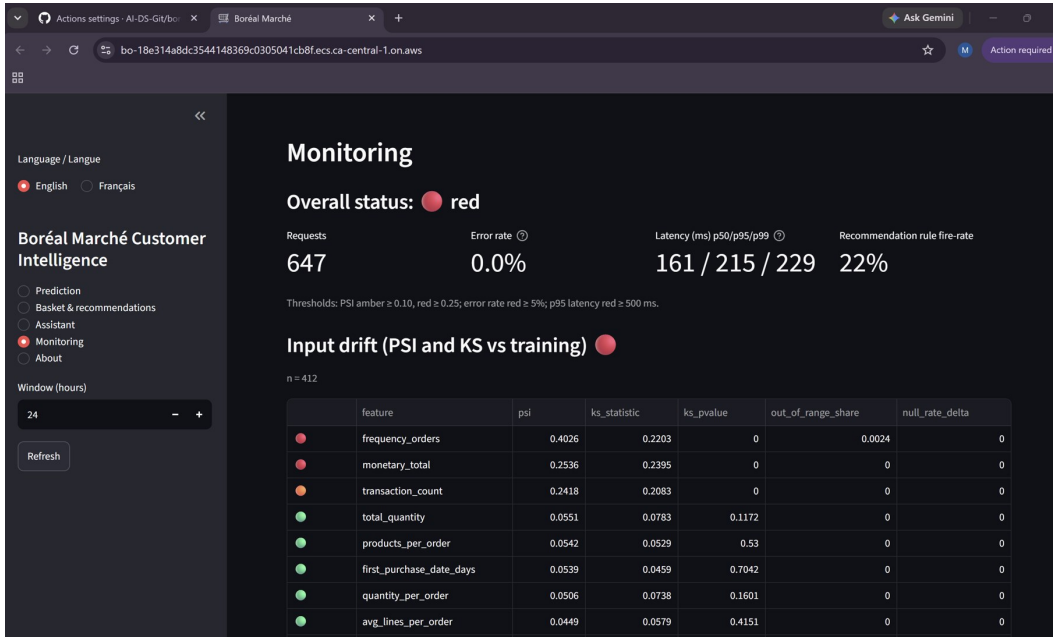
Result	Value	Baseline
Head A held-out PR-AUC / ROC-AUC (788 customers, evaluated once)	0.8753 / 0.7301	CV PR-AUC: dummy 0.673; one-rule heuristic 0.710; logistic regression 0.799
Head A expected profit (held-out)	$1,067	Blanket coupon $235; no action $0
Head B hit-rate@5 / catalogue coverage	0.404 / 3.15%	Popularity 0.312 / 0.32%
Head C golden set (40 questions, EN/FR, in-sample)	Tools 97.5%, refusals 97.5%, citations 95%	Real LLMs not yet measured
Live POST /predict, median / p95 (from a laptop)	109 ms / 141 ms	265 ms / 308 ms before one change
CI on every change	lint, types, 60 tests (70% gate), Docker smoke test, secret scan	Branch protection on main

Negative results kept in: Gaussian Naive Bayes is no better than the dummy; two fairness mitigations failed; the random forest scored higher than the shipped model on the held-out slice and I did not switch, because that would have used the test set for selection.

Deployed, watched and honest about it

Surveillance



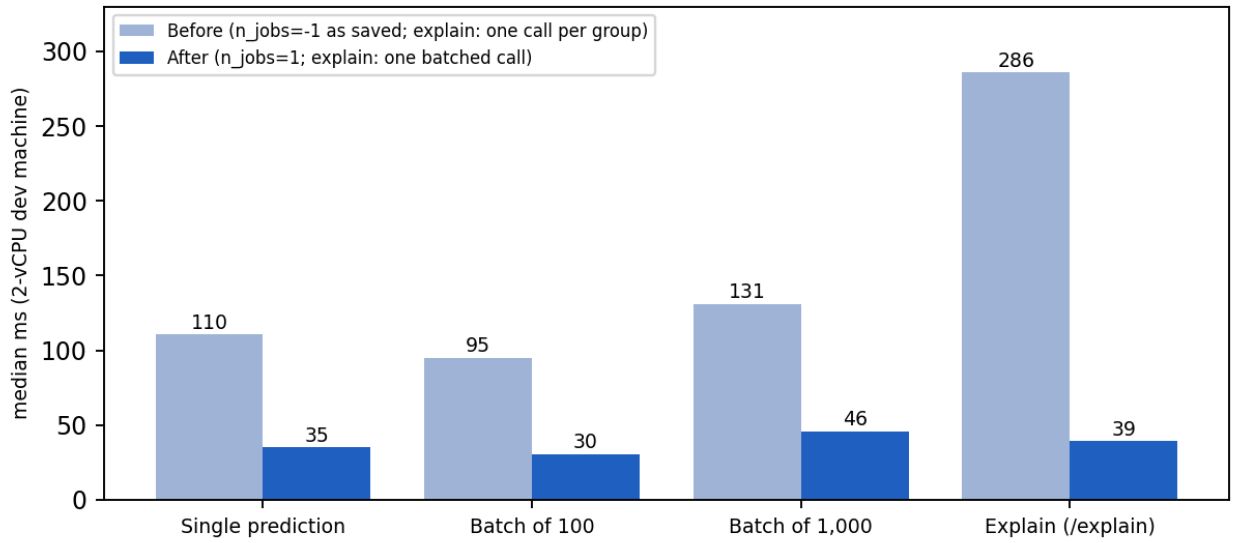


Left: normal traffic, green. Right: drifted traffic, frequency and spend features red.

- PSI drift (amber 0.10, red 0.25), data quality, latency and 5xx rate; only red states raise an alert
- Labels arrive 90 days late, so the first true accuracy signal is about 24 January 2027; the page says so
- Known limit: the log is container-local and resets on every deploy; PostgreSQL is the next step

Speed and cost

Performance et coût



- The forest was saved with n_jobs=-1: a thread pool on every call. Serving it single-threaded gives identical output.
- In process: 110 ms → 35 ms. Live: 265 ms → 109 ms median.
- /explain: 230 ms median; every call inside the 500 ms alert threshold (single client)

Cost: about $45–46 a month always on (lower bound); AWS showed $5.09 for October to date. About $0.0004 per 1,000 predictions in compute; about $0.26 per 1,000 OpenAI assistant queries (estimated, no key configured). I will delete the stack after grading.

Law 25: a person approves every offer

Loi 25

- Quebec's Law 25 (s.12.1) adds duties when a decision is made **only** by automated processing: tell the person, explain on request, allow review.
- **Design consequence:** the model recommends; a staff member approves before any offer and sees the main factors. It is not a rubber stamp.
- Bilingual customer notice; 120-day log retention then deletion; model card, data card and governance register in the repository.
- Minimisation: removing country loses nothing (PR-AUC 0.8732 vs 0.8726), so it goes at the next retraining.
- Customer-facing explanations are **not** shown: they describe how the model reacts, not why a customer behaves as they do.

Read from Légis Québec and the Commission d'accès à l'information on 26 Oct 2026. Not legal advice. / Sources lues le 26 octobre 2026. Ceci n'est pas un avis juridique.

Limitations and what I would do next

Limites et suite

Limits

- Public data; one 788-customer held-out slice
- Profit case rests on three unverified assumptions
- Cannot separate retained from churned among new and low-spend customers
- Real LLM and multilingual-embedding quality, SageMaker tuning and a load test: **not done**

Next

- Move the log to PostgreSQL; scheduled purge
- Collect features that separate new and low-spend customers
- Per-segment offer rates on the monitoring page
- Evaluate the assistant with a real model

Recommendation: a three-month pilot on real data with a control group, staff approval of every offer, and three end tests: profit above the blanket coupon, no group systematically left out, no red alert for a month.
Recommandation : un projet pilote de trois mois avec groupe témoin.

Backup: questions I expect

Question	Short answer
Why PR-AUC, not accuracy?	The positive class is the majority and the cost is asymmetric; PR-AUC rewards ranking the retained class and accuracy hides the low-spend failure.
Why was the held-out slice touched once?	Selection was frozen first. Random Forest scored higher on it; I did not switch.
Why a threshold of 0.82, and was it tuned on the test set?	Expected-profit sweep with a $5 coupon. D3 swept it on the held-out slice; choosing it on the calibration slice gives 0.825 and the same held-out profit ($1,067). Consistency check, not a clean test.
What if the assumptions are wrong?	One at a time the gain stays positive; all three together lose in 5 of 27 cases, hence a pilot with a control group.
How do you know there is no leakage?	Features recomputed from raw rows matched; injecting label-window features gave PR-AUC 1.000; top correlation 0.431.
Why not collaborative filtering?	No ratings; baskets only; rules are explainable and microsecond fast; coverage 3.15% is the price.
What would break first at scale?	The container-local SQLite log, then the monitoring refresh (about 1 s at 5,000 rows).
Why joblib is risky?	It unpickles: only trusted artifacts; versions pinned to the metadata.